LoCoBench Phase 2: The Data-Generation Harness

Pipeline, validation committee, resume protocol, cost model — and two corrections to the Phase-1 contracts

FactReasoner — Ideation

August 18, 2026

Abstract

Phase 1 fixed *what* LoCoBench is: five facets, 36 subject topics over four domains, four family types, six perturbation operators, a five-prompt generation pipeline with four validators, a gold schema and the metrics. It deferred the run. This document specifies *how* the corpus is produced: a resumable harness, `src/fact_reasoner/locobench/`, driven by one command, and the protocol for using it. The design target is a harness that is cheap to iterate on — it runs end to end with no LLM and no Merlin — and safe to interrupt, because a 600-item corpus at ≈65 model calls per family is a run measured in days, and the recovery move after any failure must be to type the same command again. Three things make it usable: **resume is the default**, **every acceptance threshold lives in one auditable table**, and **every gate failure is recorded with a reason rather than silently retried**. Two Phase-1 defects surfaced while encoding its contracts as executable assertions, and both would have produced checks that can never pass: the C1 ordering contract demands a strict increase in `log_partition` at a rung where the reference ladder itself shows no change, and the schema's `topic` field holds a free-text framing that cannot be checked against the hard 36-topic coverage constraint. Both are corrected here, in Section 1.3.

Contents

1 What Phase 2 delivers **2**
- 1.1 Scope . . . 2
- 1.2 What Phase 2 does *not* change . . . 2
- 1.3 Two corrections to the Phase-1 contracts . . . 2
- 1.4 Smaller inconsistencies, settled . . . 3

2 The harness at a glance **4**
- 2.1 One command . . . 4
- 2.2 The stage machine . . . 5
- 2.3 The modules . . . 5

3 The generation pipeline, stage by stage **5**
- 3.1 Topic selection . . . 5
- 3.2 A worked family . . . 5

4 Perturbation and the ladders **7**
- 4.1 Operators to calls . . . 7
- 4.2 The four ladders, as recipes . . . 7
- 4.3 The expectations the harness emits . . . 8

5 The validation committee **8**
5.1 The panel, and why the generator is excluded 8
5.2 Every threshold, in one table 9
5.3 The human subsample 9

6 Storage, resume and provenance **11**
6.1 On-disk layout 11
6.2 The resume algorithm 11
6.3 Provenance carried per item 11

7 Cost model **11**

8 Risk mitigations **14**

9 Milestones **14**

10 Summary **15**

1 What Phase 2 delivers

1.1 Scope

Phase 1 closes with the canonical statement of this phase's job:

> "Phase 2 is the generation run: topic selection, the live prompt pipeline, the validation committee, and the harness that turns `expected` blocks into assertions."

Those four parts map onto this document as Sections 3 (topic selection and the prompt pipeline), 5 (the committee), 6 (the assertions, as the `expected` blocks the harness emits) and 7 (what the run costs before committing to it, which Phase-1 R6 asks for explicitly).

The deliverable is two things: this document, and the package `src/fact_reasoner/locobench/`. The corpus it produces is **120 families × 5 rungs = 600 items**, ≈4,800 gold relations, with all 36 topics represented and a floor of three families each.

1.2 What Phase 2 does *not* change

The facets, the taxonomy, the prompt texts, the schema and the metrics are Phase-1 artefacts and are treated here as fixed. Where this document specifies a prompt, it is the Phase-1 text verbatim; the harness holds those strings as module constants and a test asserts they still match Phase 1's instruction count and placeholder set, so the two cannot drift (Section 9).

Two Phase-1 items are also deliberately left alone. `reified` stays out of scope, though the harness *records* it per item as a diagnostic, which Phase 1 §6.1 explicitly permits. And Restatement's 0.90 strength prior stays untested as a contract; what the harness does is report the distribution of mined strength on gold Restatement edges, which is the empirical question R4 actually poses.

1.3 Two corrections to the Phase-1 contracts

Encoding Phase 1's ordering contracts and coverage constraints as executable assertions surfaced two defects. Neither is a modelling error; both are places where the specification, taken literally, cannot be satisfied. They are stated first because the harness's data structures depend on the resolutions.

Defect 1 — the C1 contract is unsatisfiable at rung 1→2 for `log_partition`. Phase 1 §5.3 assigns `mean_marginal` and `log_partition` to contract C1: a strict increase across *all four* adjacent rung pairs. But the reference ladder in Phase 1 (`tab:rungs`) reports, from an exact 2^{16}-world enumeration of the AeroParts model:

Rung	Name	`mean_marginal`	$\log Z$	
1	`base`	0.607	−9.64	
2	`concession_resolved`	0.612	−9.64	*(unchanged)*

Table 1: The offending pair, from Phase 1's own reference ladder. `mean_marginal` rises; $\log Z$ does not move at all, so a contract demanding a *strict* increase in `log_partition` here cannot be met by any correct implementation.

The reason is structural, and it is the same mechanism behind Phase-1 Finding 1. The concession edit changes an edge's *weight* (p=0.80 → 0.55), not the edge *set*. The normalized log-partition readout measures the base $\log Z$ against a ceiling built by removing contradictions and a floor built by saturating them — and both bounds are constructed from the *same edge skeleton* as the base. A weight-only change can therefore leave the normalized quantity fixed, exactly as it leaves `consistency` free to invert.

Resolution. The 1→2 step becomes a *predicted invariance* for `log_partition`, asserted as $|\Delta| \leq 2\sigma$ rather than $\Delta > 2\sigma$. C1 still governs its other three adjacent pairs. Concretely, the harness emits ordering expectations **per readout per adjacent pair**, never as one chain-wide flag — which is the data-structure consequence, and the reason this defect had to be settled before writing `store.py`. Phase 1 §5.3 carries an erratum note pointing here.

Defect 2 — `meta.topic` cannot be checked against the coverage constraint. Phase 1 requires that **every one of the 36 topics** contribute families, with a floor of three. But the schema's example item carries

```
"topic":  "aviation component failure investigation"
```

which is a P1 *framing* string, not one of the 36 canonical labels (the canonical label for that item is *Civil Engineering*). As written, the hard coverage constraint is unverifiable: nothing in an item says which of the 36 buckets it belongs to.

Resolution. The item schema gains `meta.canonical_topic`, drawn from the 36, alongside the free-text `meta.framing` that P1 produced. `topics.py` is the single source of truth for the grid, and the coverage check is a query over `canonical_topic`. Both fields are kept: the framing is what makes the question specific, the canonical label is what makes coverage checkable.

1.4 Smaller inconsistencies, settled

Five further ambiguities do not change the harness's structure but would otherwise be resolved differently by different implementers. They are settled here, in the document, so the code has one reading to follow.

	The ambiguity	Settled as
1	V3's attachment: the pipeline figure draws its gate from P5, the prompt index lists it under P4	V3 runs on *every* response text, base and perturbed. It is the cheapest validator and the one whose failure mode (annotation leakage) can be introduced by a perturbation
2	V5's voter set: §7.2 says the committee runs V1/V2/V4; the figure's box fits V4/V2/V3	§7.2 is normative. V1, V2, V4 are voted; V3 is a single-model audit
3	P2's composition: instruction 3 mandates 24 claims + 2 holdings, the worked exemplar shows 17	The instruction wins; the exemplar is abbreviated for the page. The parser validates against instruction 3
4	O5's inverse is named `add_resolution` in the manifest but only `remove_resolution` in §4.2	Both registered. `remove_resolution` builds the unresolved rung, `add_resolution` the resolved one
5	Topic floor: the `tab:topics` caption says "at least one family", the allocation paragraph says three	Three. The caption understates the constraint the allocation rule actually imposes

Table 2: Ambiguities in Phase 1 settled by this document. None requires a Phase-1 edit beyond the Defect-1 erratum; they are recorded so the harness has a single reading.

2 The harness at a glance

2.1 One command

The normal way to use the harness is one command, and the recovery move after any failure is the same command again:

Command	What it does
`locobench-generate --config locobench.json`	the run; re-run to resume
`locobench-generate --dry-run --limit 2`	full pipeline, no LLM, no Merlin
`locobench-generate ... --only-topic Law`	one topic, for inspection
`locobench-generate ... --stage plan`	stop after P3
`locobench-generate ... --report`	coverage and cost, generate nothing

Three properties make that safe, and they are the harness's actual design requirements rather than nice-to-haves:

(1) **Resume is the default.** Every run reads the existing output directory first and prints what it found (`431/600 items, 12 gate failures, resuming`). Completed families are skipped; only gate failures and unstarted families are worked. There is no `--resume` flag because there is no non-resuming mode.

(2) **Dry-run covers the whole pipeline.** `--dry-run` substitutes a deterministic offline generator for the LLM and the brute-force oracle for Merlin, so the parsers, gates, ladders, schema assertions and storage all execute. A developer can exercise every code path in seconds without credentials.

(3) **Failures are recorded, not retried silently.** A family failing a gate is written to `rejected/` with the validator, the threshold and the observed value. Nothing is discarded, and the rejection rate per gate is itself a reportable finding about the prompts.

2.2 The stage machine

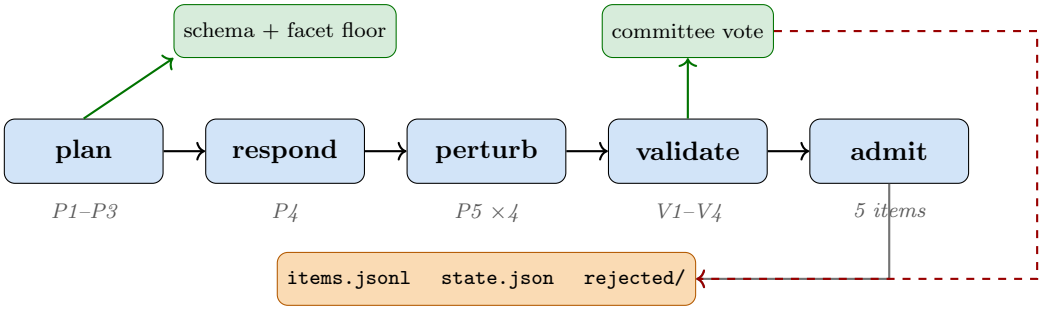

Figure 1: The five stages, the two gate points, and the store. Each stage is resumable at family granularity: `state.json` records the last stage a family completed, so an interrupted run restarts it there rather than from `plan`. The dashed edge is the rejection path — a family failing the committee vote is stored with its reason, not dropped.

2.3 The modules

What is reused rather than rebuilt. `utils.run_throttled` for bounded-concurrency fan-out — it already captures per-item exceptions instead of propagating them, which is exactly the failure isolation a long run needs; `backends.build_backend` for every model; `cli._add_backend_args` and `_backend_context` so model selection matches the other two commands; `lcs.taxonomy.COMPILE` as the authority for sense→coupling; `lcs.candidate_pairs.select` for window admission; and `experiments.mock.dry_run_patches` for the offline scoring path.

3 The generation pipeline, stage by stage

Each stage below states its inputs, the prompt it issues, what the parser must extract, the gate that admits its output, and — the part Phase 1 leaves open — **what happens on failure**. The failure column is what makes the harness a harness rather than a script.

3.1 Topic selection

`topics.py` owns the grid. `allocate(120)` gives every one of the 36 topics three families ($36 \times 3 = 108$) and distributes the remaining twelve to the topics that best support the scarce relation facets — Law, Medicine, Civil Engineering, Political Science and Ethics — because those are where exhaustive alternatives and adjudicating holdings occur naturally. The three-family floor holds without exception; the surplus is the only discretionary part, and it is recorded in the manifest so a reader can see where the corpus is deliberately denser.

Each family draws one canonical topic and asks P1 for a framing within it. Both are stored (Defect 2): `canonical_topic` for the coverage query, `framing` for provenance.

3.2 A worked family

One CONFLICT family, from topic to five admitted items, with the call count:

1. **plan.** P1 on *Civil Engineering* returns a framing about third-party component failure with competing explanations (1 call). P2 returns 26 tagged claims including an `[alt-pair]` ("no one

Module	Responsibility
`config.py`	`GenConfig` and `load_config(path)`. JSON by default (stdlib); YAML accepted only when `yaml` imports, since it is *not* a declared dependency of this project. `to_dict()` travels with the output for provenance
`topics.py`	the 36 topics × 4 domains as the single source of truth, and `allocate(n)` implementing the three-per-topic floor and the surplus rule
`prompts.py`	P1–P5, V1–V4 as module constants with {{`placeholder`}} slots, verbatim from Phase 1 Appendix A
`parse.py`	one parser per prompt output. Every parser returns `(value, error)` and never raises, so a malformed generation is a gate failure rather than a crash
`perturb.py`	O1–O6, the eleven parameterized calls, and the four family ladders as declarative rung recipes
`validate.py`	the committee, the aggregation rules, the agreement statistics, and the single `THRESHOLDS` table
`schema.py`	item and manifest validators, including the `COMPILE[sense].level1 == coupling` assertion and the real `candidate_pairs.select` call that fills window admission
`store.py`	append-only `items.jsonl`, the family manifest, `state.json`, and `rejected/`
`pipeline.py`	`generate_family(topic, cfg)` — the stage machine of Figure 1
`mock.py`	the deterministic offline generator behind `--dry-run`
`cli.py`	`locobench-generate`

Table 3: The harness modules. It is a sibling of `fact_reasoner/experiments/`, not an extension of it: that package is a stateless sweep that rewrites every cell, whereas generation is stateful, gated and resumable. The scoring sweep stays where it is.

was harmed" / "three passengers died") and two `[holding]` claims (1 call). P3 returns a JSON plan: 16 atoms, 10 relations covering all five required senses, 5 non-relations (1 call, up to 3 on regeneration).

2. **respond.** P4 realizes the plan as ≥500 words of prose (1 call).
3. **perturb.** P5 calls build rungs 0, 2, 3, 4 from the base (Section 4); rung 1 *is* the base and costs nothing. This is **7 calls** for a CONFLICT ladder rather than 4, because a single rung may compose several operator calls — summing `perturb.LADDERS` is the only reliable way to count them.
4. **validate.** V1, V2, V4 run on all five items across four committee models — the generator is excluded. V1, V3 and V4 additionally run *inline* on the base response under the generator model, as the stage's own gate. This is the bulk of the cost, and Section 7 counts it.
5. **admit.** Five items written to `items.jsonl`, one manifest entry with its ordering constraints, and the family marked complete in `state.json`.

If any rung fails, the family is rejected *whole*. A four-rung ladder cannot carry the ranking claim that is the family's only purpose, so partial families are not admitted — this is why the gate sits at family granularity and not item granularity.

Stage	Prompt	Gate	On failure
plan	P1	question is bracketed, non-empty	**retry** up to 3×, then skip the topic slot and log
plan	P2	24 claims + 2 holdings; all tags present	**retry** 3×; a missing `[alt-pair]` or `[holding]` is fatal to the family, since the rare facets depend on them
plan	P3	JSON schema-valid; ≥1 each of Alternative, Disjunction, Restatement, Concession, Precedence/Succession; every edge within 4 positions or entity-sharing; 55/45 validity	**regenerate** the plan 3×. Persistent failure rejects the family with the unmet clause named
respond	P4	≥500 words; then V1, V3, V4	**regenerate** the response, keeping the plan. V1 shortfall may instead **drop the unrecovered edges** from gold (never keep them)
perturb	P5	exactly one operator applied; length drift ≤15%; JSON diff parses	**retry** the single rung 3×; a rung that will not build rejects the family, since a ladder with a hole carries no ranking claim
validate	V1–V4	the thresholds of Table 8	**reject** to `rejected/` with validator, threshold and observed value
admit	—	schema assertion; window admission recomputed on realized text	**reject**; a schema violation is a harness bug and is surfaced loudly

Table 4: The stages and their failure policies. Three distinct responses to failure — retry the call, regenerate the artefact, or reject the family — and the choice is not arbitrary: retry for transient parse failures, regenerate when the artefact is wrong but its inputs are fine, reject when the family cannot carry its ranking claim. Only V1 has a fourth option (drop the offending edges), and only because Phase 1 mandates it: unrecovered edges must never be retained as gold.

4 Perturbation and the ladders

4.1 Operators to calls

The six operators of Phase 1 §4.2 are the vocabulary a reader needs; the harness dispatches the eleven parameterized calls the P5 prompt accepts. `perturb.py` holds the mapping as a table, so adding a call never requires touching the prompt:

4.2 The four ladders, as recipes

Each family type is a declarative list of five rung recipes. `perturb.py` holds them as data, which is what lets `store.py` emit the per-pair expectations mechanically.

Op	Meaning	Calls
o1	relabel	`wrong_sense(edge, new_sense)`, `direction_reversal(edge)`, `exhaustiveness_flip(edge)`
o2	add conflict	`inject_contradiction(atom)`, `spurious_relation(atom_a, atom_b)`
o3	break chain	`break_chain(edge)`
o4	drop relation	`drop_relation(edge)`
o5	resolve/unresolve	`remove_resolution(edge)`, `add_resolution(edge)`
o6	reorder	`shuffle_order(level)`, `ordering_only(edge)`

Table 5: The operator-to-call mapping. `add_resolution` is registered explicitly (ambiguity 4 of Table 2): the manifest names it, Phase 1 §4.2 mentions only its inverse, and the resolved rung of every CONFLICT ladder needs it.

Family	n	Rungs 0→4 (each derived from the base unless noted)
CONFLICT	55	`spurious_relation` | *base* | `add_resolution` | `add_resolution`+`drop_relation` | all conflicts dropped
CHAIN	25	`break_chain`×3 | ×2 | ×1 | *base* | coherent rewrite
ORDER	25	`shuffle_order(full)` | `(block)` | `(adjacent)` | *base* | `ordering_only` *(flat rung)*
CONTROL	15	five meaning-preserving edits: `ordering_only` and `direction_reversal` only. The required pattern is a **flat line**

Table 6: The four ladders. CONTROL is the family whose correct result is *no* trend; it is what distinguishes a coherence score from a score that tracks edit count, and it is the one a hurried implementation would omit.

4.3 The expectations the harness emits

For every family the harness writes an `expected` block per item and an `ordering_constraints` list per family. Per Defect 1 these are **per readout per adjacent pair**, and for the CONFLICT ladder they come out as:

Three readouts, four pairs, and the three rows differ: that is the content of Phase 1's per-readout contract design, and it is the reason `readout_directions` is a 3×4 structure in the schema rather than a single label per rung.

5 The validation committee

5.1 The panel, and why the generator is excluded

Five models from at least three distinct families run v1, v2 and v4 independently at temperature 0. v3 is a single-model audit (ambiguity 1 of Table 2). Per-edge labels are decided by majority, except the EXCLUSIVE label, which requires unanimity.

The constraint that shapes the implementation is **per-item generator exclusion**: the model

Adjacent pair	0→1	1→2	2→3	3→4
`mean_marginal`	increase	increase	increase	increase
`log_partition`	increase	**invariant**	increase	increase
`consistency`	increase	**decrease**	unconstrained	increase

Table 7: The CONFLICT ladder's ordering expectations. The shaded cell is Defect 1's resolution: an *invariance* assertion where Phase 1 §5.3 demanded an increase. The `consistency` row is Phase 1's Finding 1 unchanged — the predicted inversion, asserted rather than exempted. Only `mean_marginal` is monotone across the whole chain, which is why it is the headline readout. The `consistency` row still holds after that readout's two-term revision (the dip lives in its conflict term), but with a narrower margin, so more items land *indeterminate* under the 2σ rule; on an EXCLUSIVE-only family, grade its conflict term alone.

that ran P3/P4 for an item may not vote on it. This is Phase-1 R3, and it is not a formality — a model asked to recover relations from prose it wrote itself recovers its own lexical fingerprints, which inflates every Target-A number. So the committee is chosen per item, not per run, and `validate.py` takes the panel and the generator identity together:

```
committee_for(item) = [m for m in cfg.committee if m != item.provenance.planned_by]
```

With a five-model panel and one generator excluded, four models vote on any given item. The harness therefore requires $\geq$4 usable committee models to reach a majority of three, and refuses to start a run configured with fewer — a check at config-load time rather than a surprise at item 400.

The cross-generation control. The harness stratifies generators across $\geq$3 families and records `planned_by` per item, so Target-A metrics can be reported both pooled and restricted to items whose generator family differs from the miner's. Phase 1 asks for that gap to be published as a bias estimate; making it computable is a storage requirement, met by carrying `planned_by` in provenance.

5.2 Every threshold, in one table

These are the Phase-1 acceptance criteria, gathered here because a run's behaviour is entirely determined by them and they should be auditable in one place. `validate.py` holds this as a single `THRESHOLDS` dict; there are no thresholds elsewhere in the harness.

5.3 The human subsample

Three annotators on 120 items (20%), stratified to cover *all* EXCLUSIVE and CO_NECESSITY gold edges, *all* resolved Concessions, and *all* ordering-only invariance rungs, plus a random remainder. The stratification is not a convenience: those are precisely the facets with no prior gold data and the highest expected model unreliability, so uniform sampling would spend the human budget on the categories that need it least. `validate.py` exposes `stratified_sample(items, n=120)` implementing exactly that rule, so the sample is reproducible from the corpus rather than chosen by hand.

Human-versus-committee agreement is what licenses the committee as a human proxy on the other 80%, so it is reported whether or not it clears the floors.

Key	Value	Meaning, and the action on breach
Per-item admission		
`v1_coupling`	≥ 0.80	fraction of planned relations recovered with the correct coupling; breach ⇒ drop the unrecovered edges or regenerate
`v1_sense`	≥ 0.70	same, for sense
`v1_rule`	majority	of the committee must meet both rates
`v2_exclusive`	unanimity	required to assign an EXCLUSIVE gold label; split vote ⇒ ship the edge `low_agreement`
`v3_scores`	all ≥ 4	fluency, formality, organization on 1–5
`v3_spans`	empty	`leakage` and `hedging` must both be empty (`artifacts` is recorded, not gated)
`v4_coverage`	$= 1.00$	every atom `asserted`; any altered/missing/merged ⇒ regenerate
Plan structure (P3)		
`claims` / `relations` / `non_relations`	14–18 / 8–12 / 4–6	selected atoms, planned edges, distractor pairs
`validity_split`	0.55/0.45	valid to invalid, per family
`window`	4 positions	or a shared named entity; re-verified on realized text
`rare_facets`	≥ 1 each	Alternative, Disjunction, Restatement, Concession, and one of Precedence/Succession
Perturbation (P5)		
`length_drift`	$\leq 15\%$	except ± 1 sentence for `inject_contradiction` / `remove_resolution` / `break_chain`
Corpus-level, checked by `--report`		
`topic_floor`	≥ 3 families	for each of the 36 canonical topics, no exception
`none_pool`	$\geq 1{,}500$	in-window distractor pairs across the corpus
Agreement (V5), measured on the human subsample		
`kappa_coupling`	≥ 0.70	Fleiss' κ, 6-way
`kappa_sense`	≥ 0.60	Fleiss' κ, 13-way
`kappa_exhaustive`	≥ 0.55	Cohen's κ, binary; the expected weak point, reported separately
`alpha_strength`	—	Krippendorff's α, ordinal, on strength bands
Scoring margins (used by the metrics, not by admission)		
`margin`	2σ	σ over 5 re-mines of the same rung; below it a constraint is `indeterminate` and leaves the denominator

Table 8: The `THRESHOLDS` table. A facet whose agreement falls below its κ floor ships flagged `low_agreement`: excluded from headline metrics, but reported — an unreliable facet is itself a result about the taxonomy, which is why the harness must not silently drop it.

6 Storage, resume and provenance

6.1 On-disk layout

Path	Contents
`items.jsonl`	one admitted item per line, keyed by `id`
`families.json`	the manifest: rungs, operators, ordering constraints, σ
`state.json`	per family: last completed stage, gate verdicts, attempt counts
`rejected/`	one JSON per rejected family: the validator, threshold, observed value
`config.json`	the resolved `GenConfig` snapshot, for provenance

`items.jsonl` is append-only and `state.json` is rewritten atomically after each family, which is the same discipline `CoherenceRunner.assess_file` uses: the output is always a valid corpus, even if the process dies mid-run.

6.2 The resume algorithm

1. Read `state.json`. Every family with `stage == "admitted"` is done.
2. Every family in `rejected/` is retried *if* its attempt count is below the limit, and reported as permanently rejected otherwise.
3. Every family in an intermediate stage restarts *at that stage*, reusing the artefacts already stored — a family that has a validated plan does not re-plan.
4. Unstarted allocation slots are generated.
5. Print the banner, then work only what steps 2–4 identified.

The consequence worth stating: **a completed run is a fixed point**. Running the command again does nothing and costs nothing, which is what makes it safe to put in a loop or a cron job while a long generation finishes.

6.3 Provenance carried per item

Enough to answer, later, why any given gold label is what it is: which model planned the item and which wrote its response (`planned_by`), which models recovered each edge and how they voted (`recovered_by`, `exhaustiveness_votes`), whether a human confirmed it, the per-validator scores, and the `window_admission` verdict computed by the real `candidate_pairs.select` call rather than assumed. This is what makes the cross-generation bias control of Section 5 computable after the fact.

7 Cost model

Phase-1 R6 asks that the run be costed before committing to 600 items. The figures below are *measured from the harness*, not estimated: call counts come from summing `perturb.LADDERS` and instrumenting a full `--dry-run` over one family of each type, and token volumes come from counting the rendered prompt and the mock response for every prompt identifier. This matters, because the hand-derived breakdown in an earlier draft of this section was wrong in two places — it assumed a flat 4 P5 calls per family and counted V2 per item — and the corrected total is lower, not higher.

A scope gap the costing exposed. Counting V3 calls revealed that the harness runs the inline validators on the *base* response only — once per family — whereas ambiguity 1 of Section 1.3 resolves V3's attachment as *every* response text, base and perturbed. A perturbed rung is therefore

Stage	Per family	Corpus	Note
P1 question	1	120	per family
P2 claims	1	120	
P3 plan	1	120	up to 3 on regeneration
P4 response	1	120	up to 3 on V1/V3/V4 failure
P5 perturbation	7 / 4	720	**7** for CONFLICT and CHAIN, **4** for ORDER and CONTROL: a rung may compose several operator calls
V1 recovery (inline)	1	120	base response, generator model
V3 audit	1	120	base response, single-model
V4 coverage (inline)	1	120	base response, generator model
V1 recovery (panel)	20	2,400	5 items × 4 non-generator voters
V4 coverage (panel)	20	2,400	
V2 exhaustiveness	12	1,440	3 conflict edges × 4 voters, *per family* — not per item
Generation + inline	**14 / 11**	**1,560**	20% of the run
Committee	**52**	**6,240**	80% of the run
Total	**66 / 63**	**7,800**	before retries; ≈8,970 at a 15% retry allowance

Table 9: Measured LLM calls. The first **Per family** figure is CONFLICT/CHAIN, the second ORDER/CONTROL. **The committee outweighs generation four to one**, so committee size is the first knob to turn if the budget binds, and caching validator responses per (item, model, validator) is worth more than any generation optimization.

admitted today without its own naturalness and leakage audit, which is exactly the text most likely to carry an artifact, since a perturbation operator rewrote it. Closing the gap means running V1/V3/V4 inline on all five responses instead of one: generation rises from 1,560 to 3,000 calls, the corpus total from 7,800 to **9,240** (+18%), and the Opus-5 projection from $129.78 to $149.16 ($171.54 with retries). At that price the spec-complete behaviour is plainly worth buying, so this is recorded as an implementation task rather than a design change — and it is a good illustration of why the cost model is worth building from measurements: the discrepancy was invisible in the hand-derived table, which simply asserted 5 V3 calls per family.

Merlin. Scoring is 5 calls per item for the three in-scope readouts (a shared base MAR and PR, plus `consistency`'s U-chain MAR and `log_partition`'s ceiling PR and floor MAP), so ≈3,000 invocations over the corpus. Second-order next to the LLM cost, and free during development because the brute-force oracle substitutes for small networks.

Dry-run first. The protocol is: get `--dry-run` green over the full pipeline, then one live family, and only then scale. The dry run exercises every parser, gate, ladder and schema assertion at zero cost, which means the expensive run starts from a harness whose logic has already been debugged.

	Calls	In/call	Out/call	Input	Output
P1	120	480	30	57,600	3,600
P2	120	918	310	110,160	37,200
P3	120	1,175	910	141,000	109,200
P4	120	1,549	913	185,880	109,560
P5	720	2,476	993	1,782,720	714,960
V1	2,520	1,527	341	3,848,040	859,320
V2	1,440	1,249	1	1,798,560	1,440
V3	120	1,195	40	143,400	4,800
V4	2,520	1,388	412	3,497,760	1,038,240
Total	**7,800**			**11.57 M**	**2.88 M**
Cost at list price				**As counted**	**+15% retry**
Claude Opus 5 ($5 / $25 per MTok)				$129.78	$149.25
Claude Sonnet 5 ($3 / $15)				$77.87	$89.55
Claude Haiku 4.5 ($1 / $5)				$25.96	$29.85

Table 10: Measured token volumes and projected cost for the full 600-item corpus. **The whole run is a two-figure to low-three-figure dollar amount**, which reframes the design question: the binding constraint is wall-clock and human adjudication time, not spend, so there is no reason to economize on committee size or retry budget. V2 costs almost nothing on output (one letter) but is the second-largest input consumer, so it is the clearest candidate for prompt caching. Input tokens are exact; *output tokens are the soft figure* — they are taken from the mock's synthetic responses, so treat them as an order-of-magnitude floor and re-measure after the first live family.

8 Risk mitigations

Phase 1 §9 lists six risks. Each maps to concrete harness behaviour rather than an intention.

	Risk	What the harness does
R1	LLM-generated exhaustiveness is unreliable	v2 unanimity for EXCLUSIVE; 100% human coverage of those edges via `stratified_sample`; `low_agreement` flagging. *Fallback if* $\kappa < 0.55$*:* `perturb.py` can seed Alternative edges from structurally guaranteed numeric/polarity complements, where exhaustiveness follows from negation rather than world knowledge
R2	gold outside the candidate window is unmineable	P3 constrains planned positions; `schema.py` runs the *real* `candidate_pairs.select` on realized text and stores the verdict; v4's `position` output re-verifies post hoc and breaches regenerate. Optional `long_range` subset is a config flag, off by default
R3	self-generation bias	per-item generator exclusion (Section 5); generators stratified over $\geq$3 families; `planned_by` stored so the pooled-versus-cross-generation gap is computable. *Plus* a 50-item human-authored anchor slice with no LLM at any stage
R4	Restatement's 0.90 prior is unreachable	not asserted. The harness reports the distribution of mined strength on gold Restatement edges, which is the answerable question
R5	the MLN formulation cannot be scored	Target B is MRF-only; the config rejects `formulation="mln"` at load time rather than failing at scoring
R6	generation cost	Table 9, and the dry-run-first protocol. `--report` prints projected cost from the current state before any new work

Table 11: Risks to harness behaviour. R3's anchor slice is the one addition Phase 1 marks *recommended yes* but does not require: 50 items authored by hand, no LLM in any stage, as an unbiased reference point against which the generated corpus can be compared. The nine existing `data/lcs` fixtures are the precedent for the format.

9 Milestones

Five gates, each with an exit criterion that can be checked rather than judged. The point of the sequence is that every expensive step is preceded by a cheap step that would have caught the same class of bug.

What M1 protects. The prompt-drift test deserves its place in an exit criterion because the failure it catches is silent: if Phase 1's Appendix A and the harness's `prompts.py` diverge, the

	Milestone	Exit criterion
M1	harness green offline	`--dry-run --limit 2` produces two schema-valid families; running it twice does zero work the second time; the prompt-drift test passes (each constant still matches Phase 1's instruction count and placeholder set)
M2	one live family	one CONFLICT family admitted end to end against real backends, all four gates passed, and its five items score in the expected order on the three readouts
M3	one topic	three families on a single topic, exercising the floor rule; per-facet counts recorded; the CONTROL family shows a flat line
M4	pilot	20 families across $\geq$8 topics and $\geq$3 generators; human agreement measured on a proportional subsample; every κ either clears its floor or its facet is flagged
M5	full corpus	120 families, all 36 topics at $\geq$3, the `none` pool $\geq$1,500, validity split within tolerance of 55/45, and the rejection rate per gate reported

Table 12: Milestones. M4 is the decision point: it is the first stage at which the R1 question — whether EXCLUSIVE is a defensible facet at all — can be answered with data, and it costs 20 families rather than 120 to get there.

corpus is generated by prompts that no document describes. Asserting the instruction count and placeholder set in a test makes the two files each other's spec.

10 Summary

1. **One command, resumable by default.** No `--resume` flag, because there is no non-resuming mode; a completed run is a fixed point.
2. **A sibling package**, `fact_reasoner/locobench/`, reusing `run_throttled`, `build_backend`, the shared backend CLI args, `COMPILE` and `candidate_pairs.select` — not a fork of the scoring sweep, which has different requirements.
3. **Every threshold in one table** (Table 8), and nowhere else in the harness.
4. **Failures are data.** Gate rejections are stored with reasons; the per-gate rejection rate is a finding about the prompts, not noise to be suppressed.
5. **Validation dominates cost nine to one**, so the committee is the budget knob and validator caching is the optimization that matters.
6. **Two Phase-1 defects corrected** — an unsatisfiable `log_partition` contract at rung 1→2, and a topic field that could not be checked against the coverage constraint. Both were found by writing the assertions down, which is the argument for having done so before the run rather than during it.

References

[Phase1] FactReasoner — Ideation. *LoCoBench: A Benchmark for Logical-Coherence Scorers. Phase 1 — facets, error taxonomy, generation prompts, gold schema and metrics.* `docs/ideation/coherence/benchmark/locobench_phase1.tex`.

[LoReFact] Q. Xie, W. Zheng, X. Shen, and R. Xia. *LoReFact: Bridging the Logic Gap in Fact-Checking.* Findings of the ACL: ACL 2026, pages 6974–6996, 2026.

[MRFDeepDive] FactReasoner — Ideation. *The Markov Random Field Formulation of Logical Coherence.* `docs/ideation/coherence_mrf_deepdive.tex`.

[FactReasoner] R. Marinescu et al. *FactReasoner: A Probabilistic Approach to Long-Form Factuality Assessment for Large Language Models.* arXiv:2502.18573, 2025.